

模块 4：Action — 执行的策略空间

模块概述

核心问题：决定了做什么之后，如何去做？

Planning 解决的是“做什么”和“先做哪个”的问题，而 Action 解决的是“怎么做”——Agent 通过什么机制将意图转化为对外部世界的实际变更。

这看似是个“实现细节”，实则是 Agent 系统设计中最关键的架构决策之一。选择不同的 Action 策略，直接决定了系统的能力边界、安全模型和工程复杂度。

本模块覆盖三大 Action 策略及其配套基础设施：

策略	代表系统	核心思路	适用场景
Tool Use	Claude Code, ChatGPT	模型生成结构化 JSON 调用预定义工具	生产级系统，需要审计和权限控制
CodeAct	OpenDevin, SWE-agent (部分)	模型编写并执行代码来完成操作	复杂逻辑、灵活探索
直接生成	Copilot, Aider	模型直接输出代码/diff，不调用工具	简单任务、代码补全

理解这三种策略的设计取舍，是构建或选择编程 Agent 的核心能力。

4.1 策略 1：Tool Use (JSON 函数调用)

Claude Code 的核心方法

Tool Use 是当前主流编程 Agent 的首选 Action 策略。其核心思路极其简洁：**模型不直接操作外部世界，而是生成结构化的 JSON 指令，由运行时（runtime）解析并执行。**

Tool Use 协议：完整的工作流程

Step 1: 工具定义 (在 System Prompt 中)

- 工具名称、描述、参数 JSON Schema
- 告诉模型"你有哪些工具可用"



Step 2: 模型推理

- 根据用户请求 + 当前上下文
- 决定调用哪个工具、传什么参数
- 生成结构化 JSON



Step 3: 运行时执行

- 解析 JSON → 参数验证 → 权限检查 → 执行
- 返回工具执行结果



Step 4: 结果注入

- 工具结果作为新消息添加到对话上下文
- 模型根据结果决定下一步

Claude API 中的实际工具定义

以 Claude Code 的 Read 工具为例，工具定义通过 API 的 `tools` 参数传递给模型：

```
{
  "name": "Read",
  "description": "Reads a file from the local filesystem. The file_path parameter must be an absolute path. By default, it reads up to 2000 lines from the beginning of the file.",
  "input_schema": {
    "type": "object",
    "properties": {
      "file_path": {
        "type": "string",
        "description": "The absolute path to the file to read"
      },
      "offset": {
        "type": "number",
        "description": "The line number to start reading from"
      },
      "limit": {
        "type": "number",
        "description": "The number of lines to read"
      }
    }
  },
  "required": ["file_path"]
}
```

```
}  
}
```

模型在推理后生成工具调用：

```
{  
  "type": "tool_use",  
  "id": "toolu_01ABC123",  
  "name": "Read",  
  "input": {  
    "file_path": "/Users/dev/project/src/auth.py"  
  }  
}
```

运行时执行后，结果以 `tool_result` 消息返回：

```
{  
  "type": "tool_result",  
  "tool_use_id": "toolu_01ABC123",  
  "content": "1→import hashlib\n2→import jwt\n3→\n4→class AuthManager:  
\n5→    def __init__(self, secret_key):\n6→        self.secret_key =  
secret_key\n..."  
}
```

Claude Code 的工具分类体系

Claude Code 的工具集经过精心设计，覆盖了编程 Agent 的核心操作：



```
□ 交互 (Interaction)
  └─ AskUserQuestion 向用户提问
```

设计哲学：每个工具都是原子操作。 `Read` 只读不写，`Edit` 只做精确替换，`Bash` 不限具体命令但提供沙箱。这种原子化设计带来三个好处：

1. **可组合性**：复杂操作由多个原子操作组合而成
2. **可审计性**：每一步操作都有明确的输入/输出，用户可以逐步审查
3. **可控权限**：可以对每个工具独立设置权限策略

为什么 Edit 不是 “Write the whole file” ？

一个微妙但重要的设计决策：Claude Code 的 `Edit` 工具使用**精确字符串替换**而非全文件重写。

```
{
  "name": "Edit",
  "input": {
    "file_path": "/project/src/auth.py",
    "old_string": "    def verify_token(self, token):\n        return\n    jwt.decode(token, self.secret_key)",
    "new_string": "    def verify_token(self, token):\n        try:\n            return jwt.decode(token, self.secret_key,\n                algorithms=['HS256'])\n        except jwt.ExpiredSignatureError:\n            raise AuthError('Token has expired')"
  }
}
```

为什么这比 “把整个文件内容发给你，你修改后发回来” 更好？

维度	字符串替换 (Edit)	全文件重写 (Write)
Token 开销	低：只传变更部分	高：传输整个文件
错误风险	低：不触碰未修改部分	高：可能“幻觉”其他行
可审计性	高：diff 一目了然	低：需人工比对
冲突检测	内建：old_string 不匹配则失败	无：静默覆盖

这正是 Tool Use 策略的力量——通过**精心设计工具接口**，让模型更不容易犯错。

Tool Use 的优势与局限

优势：

- **结构化与可审计**：每次调用都是 JSON，可以记录、回放、分析
- **权限可控**：可以对每个工具设置 allow/deny/ask 策略
- **类型安全**：参数有 JSON Schema 校验，格式错误在执行前就能捕获
- **并行友好**：独立的工具调用可以并行执行（Claude 支持 parallel tool use）

局限：

- **每次调用有开销**：每个 tool call 都需要一次 API 往返（模型生成 → 执行 → 结果返回）
- **格式刚性**：复杂的条件逻辑难以用 JSON 表达
- **工具组合爆炸**：当需要很多步时，模型可能在工具序列中“迷路”
- **无法表达控制流**：没有 if/else、循环、变量——每步都是独立的

4.2 策略 2：CodeAct（代码执行）

用编程语言作为 Action 语言

CodeAct (Wang et al., 2024) 提出了一个激进但优雅的思路：**既然模型擅长写代码，为什么不让代码本身成为 Action 的载体？**

与 Tool Use 让模型生成 JSON 不同，CodeAct 让模型编写并执行完整的 Python/Shell 代码来完成操作。

核心对比：JSON Tool Call vs. Code Execution

考虑一个简单任务：“找到 src/ 目录下所有包含 TODO 的 Python 文件，并统计每个文件的 TODO 数量”。

Tool Use 方式（需要多步）：

```
Step 1: 调用 Grep 工具
{
  "name": "Grep",
  "input": {
    "pattern": "TODO",
    "path": "src/",
```

```
        "glob": "*.py",
        "output_mode": "count"
    }
}
```

→ 返回结果

Step 2: 模型解析结果, 可能需要再调用 Read 查看详情

Step 3: 模型汇总并输出

CodeAct 方式（一步完成）：

```
import os
import re

results = {}
for root, dirs, files in os.walk("src/"):
    for f in files:
        if f.endswith(".py"):
            path = os.path.join(root, f)
            with open(path) as fh:
                content = fh.read()
                count = len(re.findall(r"#.*TODO", content))
                if count > 0:
                    results[path] = count

# 按 TODO 数量排序输出
for path, count in sorted(results.items(), key=lambda x: -x[1]):
    print(f"{path}: {count} TODOs")
```

一段代码，一次执行，直接得到结果。

CodeAct 论文的核心发现

Wang et al. (2024) 的 CodeAct 论文对比了三种 Action 策略在多种 Agent 任务上的表现：

CodeAct 实验结果概要			
任务类型	Tool Use	Text	CodeAct
信息检索+推理	68.2%	42.1%	73.5%
数据处理	55.0%	30.0%	70.0%
复合操作	40.0%	20.0%	60.0%

关键发现：

1. CodeAct 在需要复杂逻辑的任务上优势明显
2. Tool Use 在简单的单步调用任务上与 CodeAct 接近
3. 纯文本输出 (Text) 在需要精确操作时表现最差

为什么 CodeAct 在复杂任务上更强？

1. 完整的编程语言表达力

```
# CodeAct 可以使用变量、条件、循环、异常处理
# Tool Use 的 JSON 无法表达这些控制流

files_modified = []
for file_path in target_files:
    content = open(file_path).read()
    if "deprecated_api" in content:
        new_content = content.replace("deprecated_api()", "new_api()")
        with open(file_path, "w") as f:
            f.write(new_content)
        files_modified.append(file_path)

if not files_modified:
    print("No files needed modification")
else:
    print(f"Modified {len(files_modified)} files: {files_modified}")
```

2. 中间结果可直接复用

在 Tool Use 中，每步工具调用的结果需要通过上下文传递给模型，模型再“理解”后决定下一步。在 CodeAct 中，中间结果是 Python 变量，直接在代码中使用——零损耗、零歧义。

3. 减少 API 往返

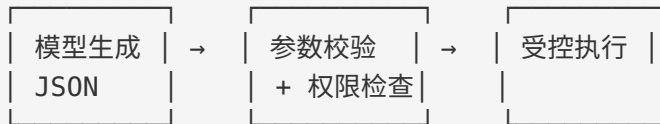
一段代码可以完成多步操作，无需多次 API 调用。这不仅提高速度，还减少了模型在多步推理中“迷路”的风险。

CodeAct 的代价

安全性是最大挑战：

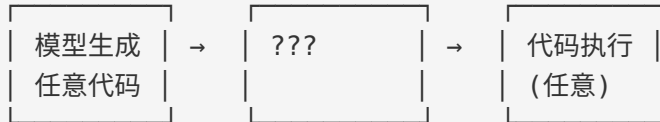
CodeAct 的安全风险

Tool Use:



→ 攻击面小：只能调用预定义的工具

CodeAct:



→ 攻击面大：可以执行任意代码

可能的恶意行为:

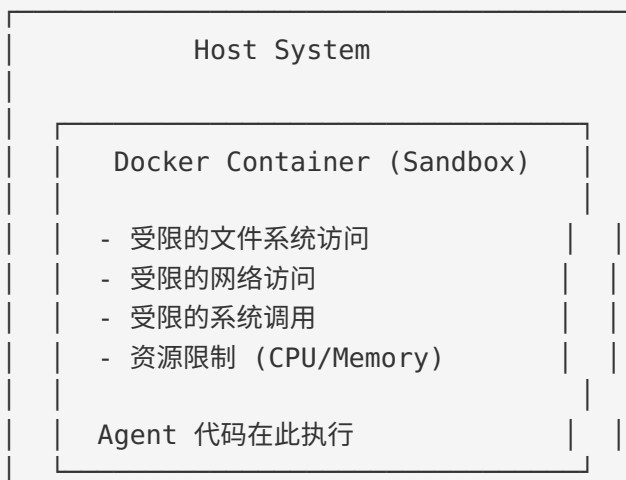
- `os.system("rm -rf /")`
- 读取 `~/.ssh/id_rsa` 并外传
- 安装后门程序
- 发起网络请求到恶意服务器

审计困难:

- Tool Use 的 JSON 调用可以逐条审查，语义明确
- CodeAct 的代码可能包含复杂逻辑，审查成本高
- 代码中可能有“看似无害但实际危险”的操作（如 `eval()`、`pickle.loads()`）

沙箱要求:

CodeAct 必须在严格的沙箱环境中执行。OpenDevin 使用 Docker 容器隔离:



混合方案：实践中的折中

现实中的系统往往不是纯粹的 Tool Use 或纯粹的 CodeAct，而是混合使用。

Claude Code 的 `Bash` 工具就是一个精妙的混合设计：

```
{
  "name": "Bash",
  "input": {
    "command": "cd /project && python -c \"import ast; tree =
ast.parse(open('src/main.py').read()); print([node.name for node in
ast.walk(tree) if isinstance(node, ast.FunctionDef)])\""
  }
}
```

形式上是 Tool Use（JSON 调用 Bash 工具），内容上是 CodeAct（参数中包含可执行代码）。这种设计兼顾了：

- **Tool Use 的可控性**：Bash 工具的执行受权限系统管控
- **CodeAct 的灵活性**：命令内容可以是任意 shell/Python 代码

4.3 策略 3：直接生成

不调用工具，直接输出代码

最简单的 Action 策略：模型直接输出代码文本，由用户或 IDE 决定如何应用。

Copilot 模式：内联代码生成

GitHub Copilot 是“直接生成”策略的典型代表：

```
用户在 IDE 中输入：
def fibonacci(n):
    """Return the nth Fibonacci number."""
    | ← 光标在此
```

```
模型直接续写：
    if n <= 1:
```

```
    return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

模型不调用任何工具，不执行任何命令，只是根据上下文生成代码文本。

特点： - **零开销**：没有工具调用的 JSON 序列化/反序列化 - **实时性**：适合每秒数次的代码补全场景 - **被动式**：模型不主动做任何事，只在用户需要时提供建议

Aider 模式：结构化 Diff 输出

Aider 采用了一种更结构化的直接生成方式——模型输出“搜索/替换”格式的 diff：

```
user: Fix the bug in the login function

assistant:
<<<<<<< SEARCH
def login(username, password):
    user = db.query(f"SELECT * FROM users WHERE name='{username}'")
    if user and user.password == password:
        return create_session(user)
=====
def login(username, password):
    user = db.query("SELECT * FROM users WHERE name = %s", (username,))
    if user and verify_password(password, user.password_hash):
        return create_session(user)
>>>>>>> REPLACE
```

Aider 解析这种格式并自动应用到文件中。注意这**不是** Tool Use——模型不调用 `Edit` 工具，而是在自然语言回复中嵌入了特殊格式的代码块。

直接生成的适用场景与局限

适合的场景：

- 单文件、单函数的修改
- 代码补全和续写
- 生成新文件/新函数
- 用户已明确知道需要什么，只需要代码实现

不适合的场景：

- 需要先读取现有代码才能修改的任务
 - 模型不知道文件当前内容，可能“幻觉”

- ❑ 需要运行测试验证的任务
 - 无法执行验证，只能"生成并祈祷"
- ❑ 跨多文件的复杂重构
 - 无法追踪文件间的依赖关系
- ❑ 需要环境信息的任务（安装的包版本、OS 配置等）
 - 无法探测运行环境

三种策略的全面对比

维度	Tool Use	CodeAct	直接生成
表达力	中（受限于预定义工具）	高（完整编程语言）	低（只能输出文本）
安全性	高（参数校验+权限控制）	低（需严格沙箱）	高（不执行代码）
可审计性	高（结构化 JSON 日志）	中（需审查代码逻辑）	低（混在自然语言中）
验证能力	有（可调用测试工具）	有（可运行测试代码）	无（只输出，不验证）
API 效率	低（每步一次往返）	高（一次执行多步）	最高（无 API 调用）
适用系统	Claude Code, ChatGPT	OpenDevin, CodeAct	Copilot, Aider
工程复杂度	中	高（沙箱+安全）	低

4.4 工具设计原则

什么样的工具接口对模型友好？

工具设计不是“能用就行”——设计不良的工具接口会让模型频繁出错、浪费 token、甚至完全无法完成任务。SWE-agent（Yang et al., 2024）的核心贡献之一就是提出了 **Agent-Computer Interface (ACI)** 的概念：工具接口应该为 AI Agent 专门优化，而不是照搬给人类用的命令行。

原则 1：简单、一致的接口

好的设计：参数少、语义清晰

```
// Claude Code 的 Edit 工具：3 个参数，语义明确
{
```

```

"name": "Edit",
"input": {
  "file_path": "/src/main.py",
  "old_string": "print('hello')",
  "new_string": "print('hello, world')"
}
}

```

差的设计：参数多、选项复杂

// 反面示例：过度参数化的编辑工具

```

{
  "name": "EditFile",
  "input": {
    "file_path": "/src/main.py",
    "mode": "replace",          // 还有 "insert", "delete", "append"
    "search_mode": "exact",     // 还有 "regex", "fuzzy", "line_range"
    "search_string": "print('hello')",
    "replace_string": "print('hello, world')",
    "case_sensitive": true,
    "max_replacements": 1,
    "backup": true,
    "encoding": "utf-8",
    "line_ending": "auto"
  }
}

```

模型不是人类——它不会“查文档”来理解复杂的参数组合。每增加一个参数，模型犯错的概率就增加。

SWE-agent 的经验数据：当工具参数从 3 个增加到 7 个时，模型的工具调用错误率从 ~5% 上升到 ~20%。

原则 2：结构化、信息丰富的输出

好的设计：返回结构化信息

```

$ grep -n "TODO" src/*.py
src/auth.py:23:    # TODO: Add rate limiting
src/auth.py:45:    # TODO: Validate token expiry
src/db.py:12:    # TODO: Connection pooling

Found 3 matches in 2 files

```

输出中包含文件名、行号、匹配内容和汇总信息。模型可以直接利用这些信息决定下一步。

差的设计：返回大量无结构原始文本

```
$ find . -name "*.py" -exec cat {} \;
```

[数千行代码不加区分地倾泻而出]

模型必须从大量文本中自行提取有用信息，容易遗漏或误解。

原则 3：适当的操作粒度

粒度过细：

```
open_file(path)
seek_to_line(line_num)
read_n_lines(n)
close_file(path)
```

- 4 步才能读一段代码
- 状态管理复杂

粒度过粗：

```
edit_project(description)
```

- 模型不知道到底会发生什么
- 无法精确控制

————— 适中 —————

```
read_file(path, offset, limit)
```

- 一步完成，参数控制精度
- 无状态，每次调用自包含

关键洞察：工具应该是**无状态的**。需要“先打开文件再读取”的有状态接口对模型极不友好——模型很容易忘记关闭文件、搞混文件句柄、在错误的状态上操作。

Claude Code 的每个工具都是无状态的一次调用：`Read` 直接传路径返回内容，`Edit` 直接传路径和替换内容完成编辑，不需要任何预置步骤。

原则 4：帮助模型恢复的错误信息

好的错误信息：说明原因 + 给出建议

```
Error: Edit failed - old_string not found in file.
```

```
The string you provided does not match any content in /src/auth.py.
This may be because:
```

1. The file has been modified since you last read it
2. The indentation doesn't match (spaces vs tabs)

Suggestion: Use the Read tool to view the current file content, then retry with the exact string.

差的错误信息：只有错误码

Error: ENOENT

模型收到“ENOENT”后可能会：- 反复重试同样的操作 - 胡乱猜测路径 - 放弃当前方案
而收到详细错误信息后，模型通常能够：- 理解失败原因 - 按建议执行恢复操作 - 成功完成任务

SWE-agent 的 ACI 设计实践

SWE-agent 不使用标准的 Unix 命令行，而是设计了一套专为 Agent 优化的自定义工具：

标准 Unix 命令 vs SWE-agent 自定义工具：

Unix 命令	SWE-agent ACI
cat file.py (输出全部内容， 长文件信息过载)	open file.py (显示带行号的 100 行窗口， 自动滚动管理)
sed -i 's/old/new/g' file.py (正则语法复杂， 容易出错)	edit 15:20 (指定行号范围精确编辑， 语法简洁， 自动显示编辑结果)
grep -rn "pattern" . (输出可能非常长， 无汇总)	search_dir "pattern" src/ (自动限制输出量， 附带匹配汇总)
find . -name "*.py" (结果可能很多， 无优先级)	find_file "*.py" src/ (限制结果数量， 按相关性排序)

SWE-agent 的实验表明：使用 ACI 优化后的工具集，Agent 在 SWE-bench 上的任务解决率从 **1.7% 提升到 12.5%**——工具接口设计的影响巨大。

反模式汇总

反模式	问题	改进方向
需要多步状态管理的工具	模型容易丢失状态	设计无状态的单次调用
输出模糊的工具	模型无法判断成功/失败	返回明确的结构化结果
参数过多的工具	模型频繁传错参数	减少必须参数，合理设默认值
“万能工具”	语义模糊，模型不知何时用	拆分为职责单一的小工具
纯文本大量输出	上下文窗口爆炸	限制输出量，提供汇总

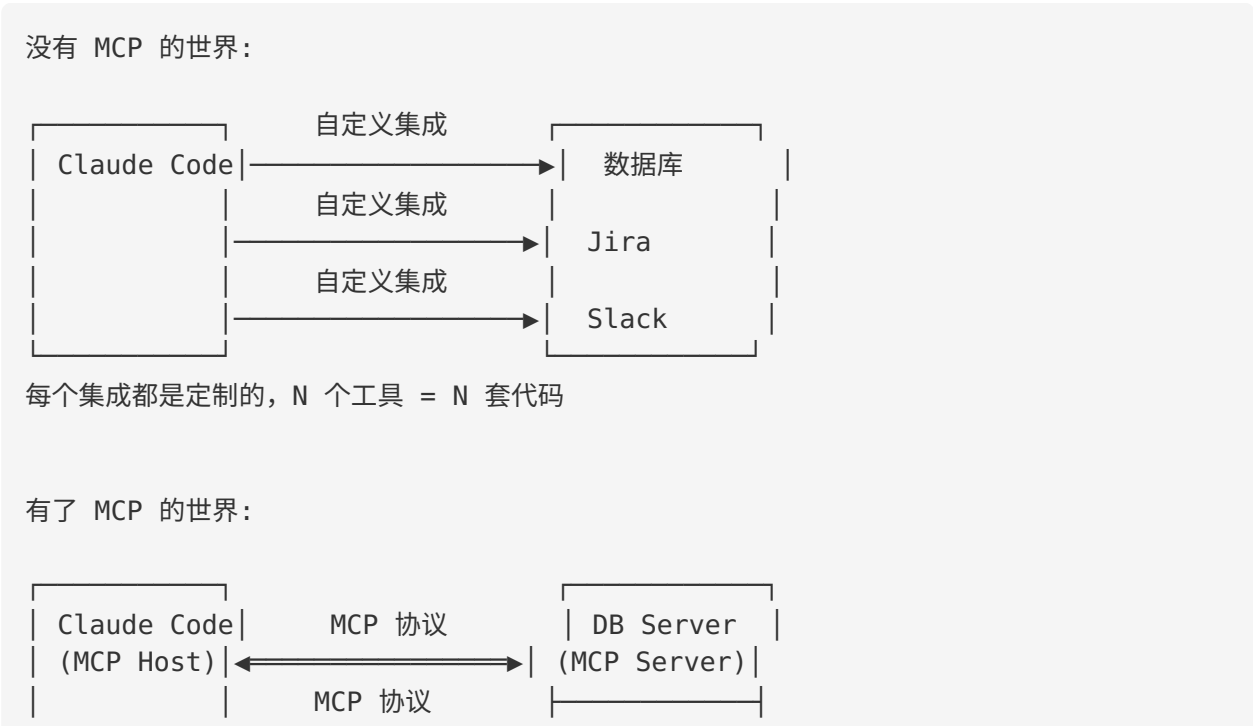
4.5 MCP 协议

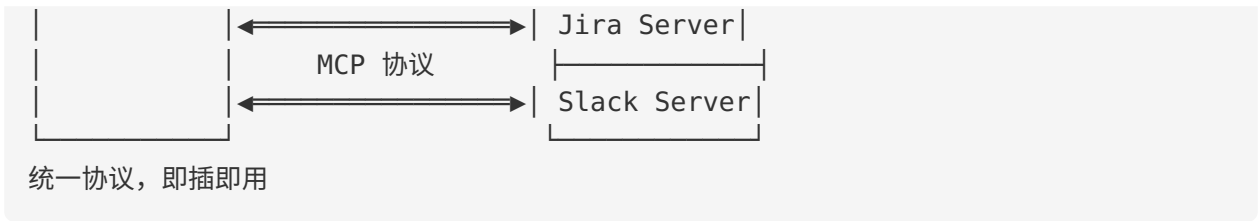
给 Agent “安装新技能” 的标准协议

当 Claude Code 内建的工具不够用时怎么办？MCP（Model Context Protocol）就是答案——它允许你为 Agent 动态添加新工具，而无需修改 Agent 本身的代码。

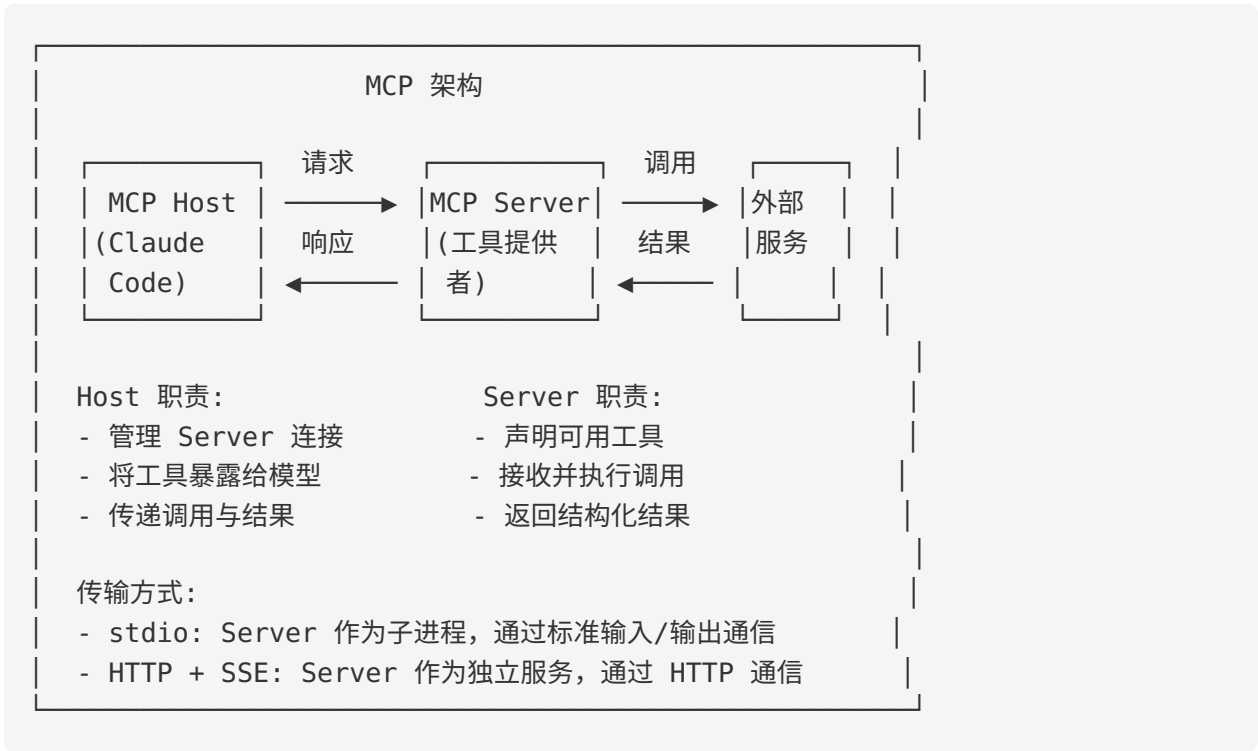
什么是 MCP

MCP 是 Anthropic 于 2024 年底推出的开放协议，定义了 **AI 模型与外部工具/数据源之间的标准通信接口**。





MCP 架构核心概念



MCP 的三大核心能力:

能力	说明	示例
Tools	模型可调用的函数	查询数据库、创建 Jira ticket
Resources	模型可读取的数据源	文件内容、API 响应
Prompts	预定义的提示词模板	代码审查模板、SQL 生成模板

构建一个 MCP Server：数据库查询工具

以构建一个 SQLite 查询 MCP Server 为例，展示完整流程：

Step 1: 项目初始化


```
mkdir sqlite-mcp-server && cd sqlite-mcp-server
npm init -y
npm install @modelcontextprotocol/sdk better-sqlite3
```

Step 2: 实现 MCP Server

```
// src/index.ts
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import Database from "better-sqlite3";
import { z } from "zod";

const DB_PATH = process.env.DB_PATH || "./data.db";

const server = new McpServer({
  name: "sqlite-query",
  version: "1.0.0",
});

// 定义工具：执行 SQL 查询
server.tool(
  "query",
  "Execute a read-only SQL query against the SQLite database",
  {
    sql: z.string().describe("The SQL query to execute (SELECT only)"),
  },
  async ({ sql }) => {
    // 安全检查：只允许 SELECT 语句
    const normalized = sql.trim().toUpperCase();
    if (!normalized.startsWith("SELECT")) {
      return {
        content: [
          {
            type: "text",
            text: "Error: Only SELECT queries are allowed for safety.",
          },
        ],
      };
    }

    try {
      const db = new Database(DB_PATH, { readonly: true });
      const rows = db.prepare(sql).all();
      db.close();

      return {
        content: [
          {

```

```

        type: "text",
        text: JSON.stringify(rows, null, 2),
    },
],
};
} catch (error) {
    return {
        content: [
            {
                type: "text",
                text: `Query error: ${error.message}\n\nPlease check your SQL
syntax and table/column names.`
            },
        ],
    };
}
}
);

```

// 定义工具：列出所有表

```

server.tool(
    "list_tables",
    "List all tables in the database with their schemas",
    {},
    async () => {
        const db = new Database(DB_PATH, { readonly: true });
        const tables = db
            .prepare(
                "SELECT name, sql FROM sqlite_master WHERE type='table' ORDER BY
name"
            )
            .all();
        db.close();

        return {
            content: [
                {
                    type: "text",
                    text: tables
                        .map((t) => `## ${t.name}\n\`\`\`sql\n${t.sql}\n\`\`\`)
                        .join("\n\n"),
                },
            ],
        };
    }
);

```

// 启动 Server

```

const transport = new StdioServerTransport();
await server.connect(transport);

```

Step 3: 在 Claude Code 中配置

在项目的 `.claude/settings.json` 中添加 MCP Server 配置：

```
{
  "mcpServers": {
    "sqlite": {
      "command": "node",
      "args": [".//sqlite-mcp-server/src/index.ts"],
      "env": {
        "DB_PATH": "./data/app.db"
      }
    }
  }
}
```

配置完成后，Claude Code 启动时会自动连接 MCP Server，模型就能使用 `query` 和 `list_tables` 工具了。

Step 4: 使用效果

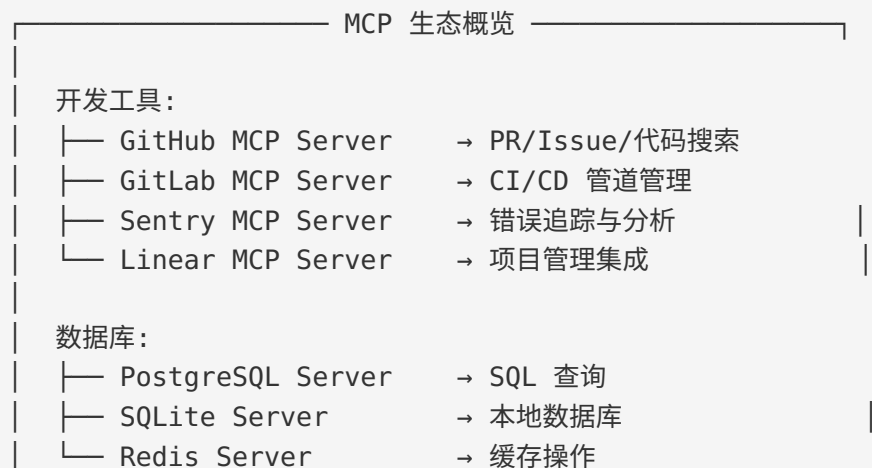
用户： "查一下最近一周注册的用户数量"

Claude Code 内部：

1. 调用 `list_tables` → 获取数据库 schema
2. 调用 `query` → `SELECT COUNT(*) FROM users WHERE created_at > datetime('now', '-7 days')`
3. 输出结果： "最近一周注册了 142 个新用户。"

MCP 生态

MCP 已经形成了丰富的工具生态：



知识/文档：

- └─ Notion MCP Server → 文档读写
- └─ Google Drive Server → 文件管理
- └─ Confluence Server → 知识库查询

基础设施：

- └─ AWS MCP Server → 云资源管理
- └─ Docker MCP Server → 容器操作
- └─ Kubernetes Server → 集群管理

通信：

- └─ Slack MCP Server → 消息发送/检索
- └─ Email MCP Server → 邮件操作

MCP 的核心价值在于**解耦**：Agent 不需要知道“怎么连接 PostgreSQL”或“怎么调用 Jira API”，只需要理解工具的语义描述和参数格式。这让 Agent 能力的扩展变成了“写一个 MCP Server”的工程问题，而非“修改 Agent 核心代码”的架构问题。

4.6 权限与沙箱

Action 的安全边界

Agent 能做的事情越多，风险就越大。一个能读写文件、执行命令、访问网络的 Agent，如果不加约束，本质上等于一个拥有用户权限的自动化脚本——出错时的破坏力是巨大的。

Claude Code 的权限模型

Claude Code 采用**分级权限**设计：

Claude Code 权限级别

Level 1: 始终允许 (Always Allow)

Read, Glob, Grep, WebSearch

→ 只读操作，不修改任何状态

Level 2: 需要用户确认 (Ask)



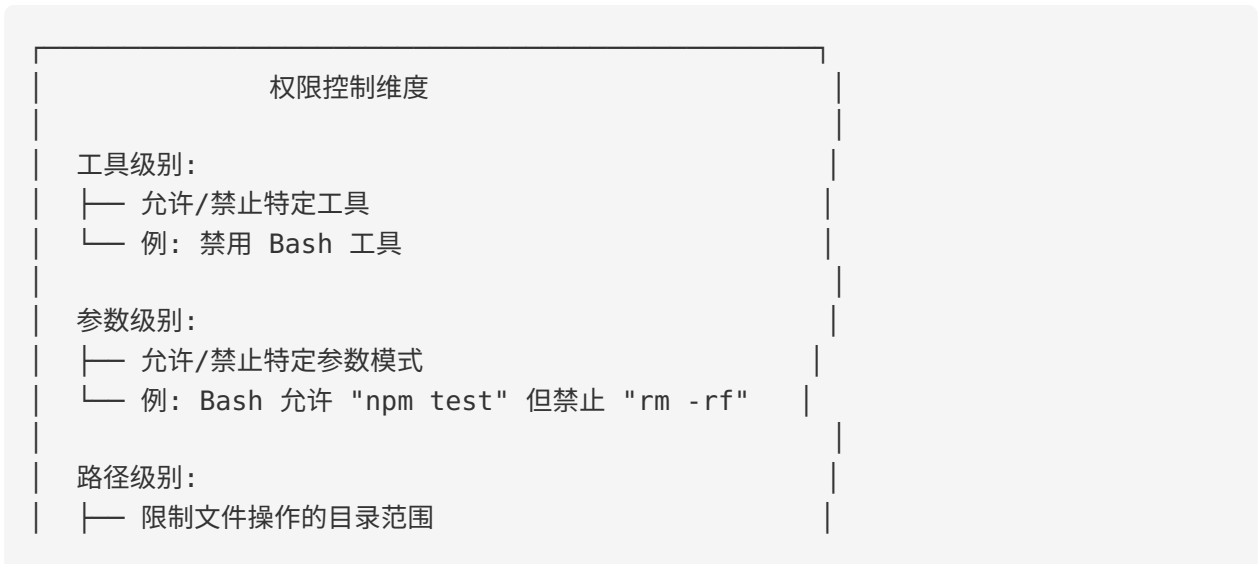
这种设计的核心理念是“默认安全，按需放开”：

用户 workflows 示例：

- 首次使用：所有写操作都需确认
Claude: [要执行 Edit] → 用户: ✓ 允许
- 建立信任后：配置自动允许常用操作
settings.json: { "allowedTools": ["Edit", "Write"] }
- 危险操作始终需确认：
Claude: [要执行 `rm -rf node_modules/`] → 用户: ⚠ 审查后决定

权限的作用域

不同维度的权限控制构成了多层防御：



└─ 例：只允许在项目目录内读写

网络级别：

└─ 限制网络访问的范围

└─ 例：只允许访问特定 API 域名

沙箱策略

更激进的安全方案是**沙箱隔离**——在受限环境中执行 Agent 的操作：

容器化隔离 (Docker)：

```
# Agent 执行环境配置
services:
  agent-sandbox:
    image: agent-runtime:latest
    security_opt:
      - no-new-privileges:true
    read_only: true
    tmpfs:
      - /tmp
    volumes:
      - ./workspace:/workspace # 只挂载工作目录
    networks:
      - restricted # 受限网络
    deploy:
      resources:
        limits:
          cpus: "2.0"
          memory: 4G
```

文件系统隔离：

Host 文件系统：

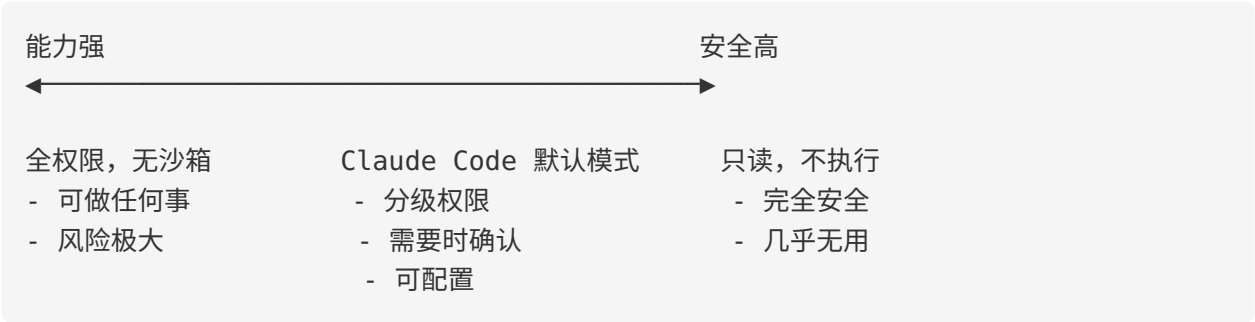
```
/
├─ home/
│   ├─ user/
│   │   ├─ .ssh/          ← Agent 不可访问
│   │   ├─ .aws/         ← Agent 不可访问
│   │   └─ projects/
│   │       └─ my-app/    ← Agent 工作目录（可访问）
│   └─ ...
└─ ...
```

Agent 可见的文件系统：

```
/workspace/                                ← 映射自 ~/projects/my-app/
├── src/
├── tests/
├── package.json
└── ...
```

能力与安全的张力

这是 Agent 系统设计中最核心的矛盾之一：



不同系统在这条线上的位置：

系统	定位	权限模型
GitHub Copilot	安全优先	只生成代码，不执行任何操作
Cursor	中间偏安全	编辑需确认，有限 shell 访问
Claude Code	中间偏能力	分级权限，可配置自动允许
OpenDevin	能力优先	Docker 沙箱内全权限
裸执行脚本	完全能力	无保护，极度危险

OpenCode 的权限设计对比

OpenCode（一个开源的 Claude Code 替代品）采用了不同的权限策略：

Claude Code 权限模型：

每个工具调用都经过权限检查

tool_call → permission_check

→ user_prompt (if needed)

→ execute

粒度：单次工具调用

OpenCode 权限模型：

基于"会话模式"的权限

mode: "safe" → 只读操作

mode: "normal" → 常规操作+确认

mode: "yolo" → 全部自动允许

粒度：整个会话

两种设计各有取舍：

维度	Claude Code (per-call)	OpenCode (per-session)
安全粒度	高：每次调用独立控制	低：整个会话一个策略
用户负担	较高：频繁确认	较低：一次设定
灵活性	高：可精细配置	中：三档模式选择
误操作风险	低	“yolo” 模式下较高

在实践中，Claude Code 通过 `settings.json` 的 `allowedTools` 配置和 Bash 命令的模式匹配，在安全性和便利性之间找到了良好的平衡。用户可以逐步建立信任，从“全部确认”过渡到“常用操作自动允许”。

关键论文导读

CodeAct: Executable Code Actions Elicit Better LLM Agents (Wang et al., 2024)

核心贡献：

提出用可执行代码（Python）替代 JSON 格式的 Action 表示，并在多种 Agent 任务上验证了这种方法的有效性。

关键实验设计：

- 在 MINT benchmark 上对比 CodeAct、Tool Use（JSON）和纯文本三种 Action 策略
- 使用相同的 base model（GPT-3.5 和 GPT-4），只改变 Action 格式
- 控制变量：相同的系统提示词结构、相同的任务描述

核心发现：

- 1. CodeAct 在需要多步推理和数据处理的任務上优势明显
- 2. Tool Use 在简单的单工具调用任务上与 CodeAct 相当
- 3. 代码作为 Action 语言有天然的“自我纠错”能力 (try/except)
- 4. CodeAct 需要的 API 调用次数更少 (一段代码 vs 多步工具调用)

对从业者的启示：

- 如果你的 Agent 需要处理复杂的数据转换和逻辑判断，考虑 CodeAct 方式
- 如果你的 Agent 面向终端用户且需要严格审计，Tool Use 更合适
- 混合方式（如 Claude Code 的 Bash 工具）可以兼顾两者优势

SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering (Yang et al., 2024)

核心贡献：

提出 Agent-Computer Interface (ACI) 的概念——工具接口应该专门为 AI Agent 设计，而不是直接复用人类的命令行工具。并通过大量消融实验验证了 ACI 设计对 Agent 性能的巨大影响。

关键实验：ACI 消融研究

实验：在 SWE-bench 上比较不同接口设计的影响

基线：	使用标准 Linux 命令行	→ 1.7% 解决率
+ 编辑优化：	替换 sed 为带行号的 edit 命令	→ 5.3% 解决率
+ 搜索优化：	替换 grep 为带上下文的 search	→ 8.1% 解决率
+ 导航优化：	添加 open/scroll 导航命令	→ 10.2% 解决率
+ 错误恢复：	添加 lint 检查和错误提示	→ 12.5% 解决率

从 1.7% 到 12.5% — 完全相同的模型，仅改变工具接口

ACI 设计的核心原则（论文总结）：

- 1. 简洁明了的命令：命令名和参数应自解释
- 2. 紧凑但信息丰富的反馈：不多不少，恰好够决策
- 3. 防误操作机制：自动 lint、编辑回滚、格式检查
- 4. 参考文档可用：命令的帮助信息可被模型查看

对从业者的启示：

- 不要直接把 Unix 命令暴露给 Agent——它们是为人类设计的
- 工具的错误信息比工具本身更重要——好的错误信息可以让 Agent 自我修复
- 投入时间设计 ACI 的 ROI 极高——可能比换一个更好的模型效果还大

实操环节

实操 1：构建自定义 MCP Server 并连接 Claude Code

目标：构建一个查询本地 SQLite 数据库的 MCP Server，让 Claude Code 能直接查询数据。

步骤：

```
# 1. 创建项目目录
mkdir my-db-mcp && cd my-db-mcp
npm init -y
npm install @modelcontextprotocol/sdk better-sqlite3

# 2. 创建测试数据库
sqlite3 test.db <<EOF
CREATE TABLE users (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT UNIQUE,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO users (name, email) VALUES
  ('Alice', 'alice@example.com'),
  ('Bob', 'bob@example.com'),
  ('Charlie', 'charlie@example.com');

CREATE TABLE orders (
  id INTEGER PRIMARY KEY,
  user_id INTEGER REFERENCES users(id),
  amount DECIMAL(10,2),
  status TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO orders (user_id, amount, status) VALUES
  (1, 99.99, 'completed'),
  (1, 149.50, 'pending'),
  (2, 29.99, 'completed'),
  (3, 199.99, 'cancelled');
EOF
```

```
# 3. 编写 MCP Server (参见 4.5 节代码)

# 4. 配置 Claude Code
# 在项目目录的 .claude/settings.json 中添加:
# {
#   "mcpServers": {
#     "mydb": {
#       "command": "node",
#       "args": [". /my-db-mcp/src/index.ts"],
#       "env": { "DB_PATH": "./test.db" }
#     }
#   }
# }

# 5. 启动 Claude Code, 验证工具可用
claude
# 输入: "查询数据库中有多少用户"
# 预期: Claude 自动调用 MCP 的 query 工具
```

验证要点:

- MCP Server 是否正确启动 (检查 Claude Code 启动日志)
- 工具是否出现在可用工具列表中
- 模型是否能正确理解工具语义并生成有效 SQL
- 错误处理是否正常 (尝试传入无效 SQL)

实操 2: 对比 Tool Use 和 CodeAct 的效果

目标: 用同一个任务分别用 Tool Use 和 CodeAct 方式完成, 对比效率和结果。

任务: 在一个 Python 项目中, 找到所有使用了已废弃 API `old_connect()` 的文件, 将其替换为 `new_connect(host, port)`, 其中 `host` 和 `port` 从配置文件 `config.yaml` 中读取。

方式 A: Tool Use (Claude Code 标准方式)

预期操作序列:

```
Step 1: Grep 搜索 "old_connect" → 找到 3 个文件
Step 2: Read config.yaml → 获取 host 和 port 值
Step 3: Read file1.py → 查看上下文
Step 4: Edit file1.py → 替换调用
Step 5: Read file2.py → 查看上下文
Step 6: Edit file2.py → 替换调用
Step 7: Read file3.py → 查看上下文
```

Step 8: Edit file3.py → 替换调用
Step 9: Bash "python -m pytest" → 运行测试验证

共计：~9 次工具调用

方式 B：CodeAct（Bash 中执行 Python 脚本）

```
# 通过 Bash 工具执行的 Python 脚本
import yaml
import re
import glob

# 读取配置
with open("config.yaml") as f:
    config = yaml.safe_load(f)
host = config["database"]["host"]
port = config["database"]["port"]

# 批量替换
modified = []
for path in glob.glob("src/**/*.py", recursive=True):
    with open(path) as f:
        content = f.read()
        if "old_connect()" in content:
            new_content = content.replace(
                "old_connect()",
                f'new_connect("{host}", {port})'
            )
            with open(path, "w") as f:
                f.write(new_content)
            modified.append(path)

print(f"Modified {len(modified)} files: {modified}")
```

共计：1 次工具调用（Bash）

对比记录表：

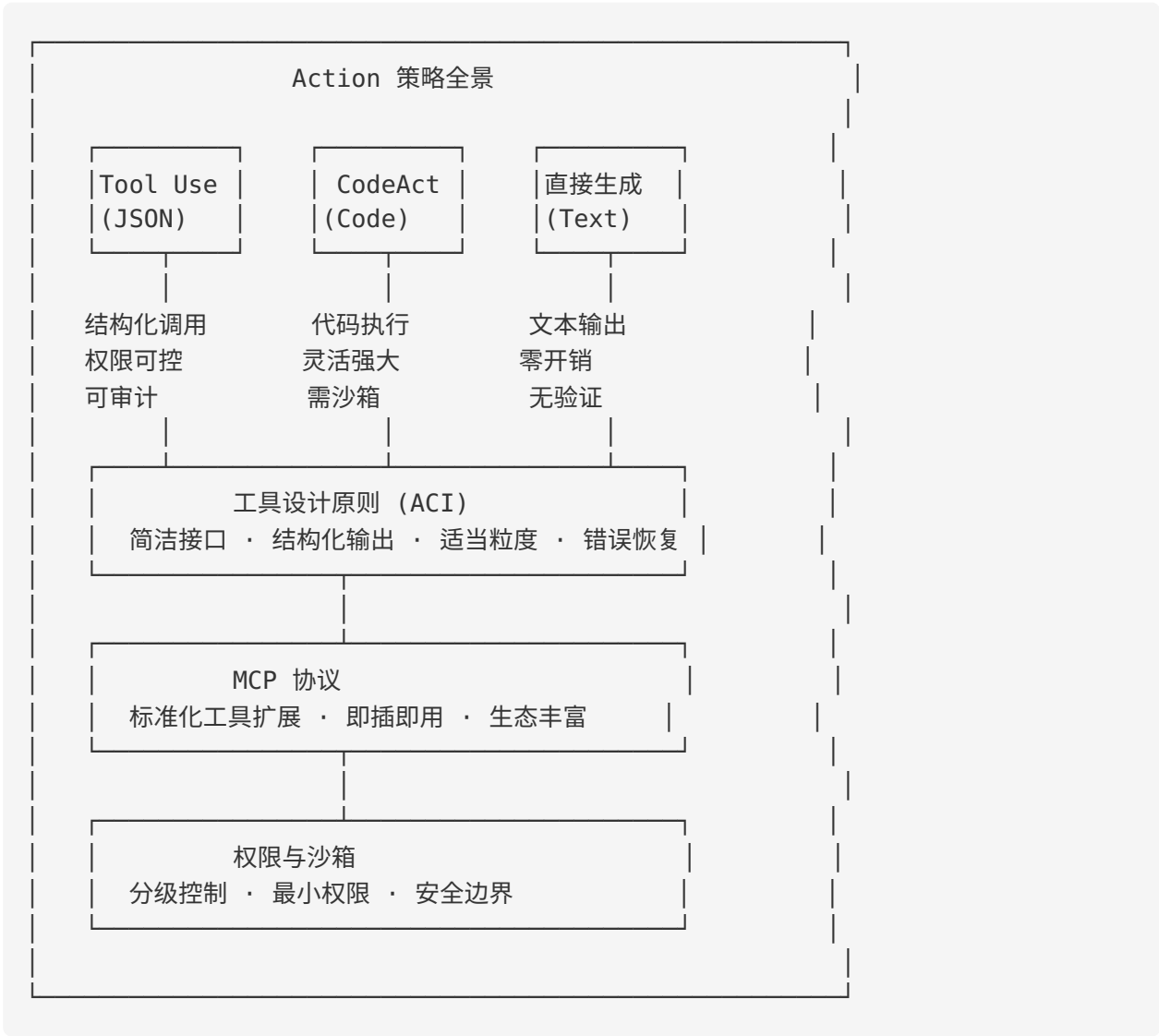
维度	Tool Use	CodeAct (via Bash)
工具调用次数		
总 Token 消耗		
完成时间		
结果正确性		

维度	Tool Use	CodeAct (via Bash)
可审计性		
错误恢复		

讨论问题： 1. 如果文件中 `old_connect()` 的调用方式不统一（有的带参数、有的在注释中），哪种方式更容易处理？ 2. 如果需要在每次替换前让用户确认，哪种方式更容易实现？ 3. 在生产环境中，你会选择哪种方式？为什么？

本模块小结

本模块探讨了 Agent 执行操作的三大策略及其配套基础设施：



核心要点：

1. **Tool Use 是生产系统的首选**——结构化、可审计、权限可控，Claude Code 证明了其有效性
 2. **CodeAct 在复杂逻辑场景有不可替代的优势**——但需要严格的沙箱保障
 3. **工具设计比模型选择更重要**——SWE-agent 的 ACI 实验证明，同样的模型配不同的工具，性能差距可达 7 倍
 4. **MCP 是工具扩展的标准答案**——让 Agent 能力扩展变成纯工程问题
 5. **安全是不可妥协的底线**——分级权限 + 沙箱隔离是保障 Agent 安全运行的两大支柱
-

思考题

1. **设计题**：如果你要为一个数据分析场景设计 Agent，你会选择 Tool Use 还是 CodeAct？需要考虑哪些因素？请设计出你的工具集（至少 5 个工具）。
2. **对比题**：Claude Code 的 `Edit` 工具使用 “old_string → new_string” 的替换方式，而 SWE-agent 使用 “行号范围” 的编辑方式。分析这两种设计各自的优劣，并思考是否存在更好的设计。
3. **安全题**：一个企业内部部署的编程 Agent 需要访问内部 GitLab、Jira 和生产数据库。请设计其权限模型，要求：(a) 开发者日常使用足够便利；(b) 不能误操作生产环境；(c) 所有操作可追溯审计。
4. **扩展题**：MCP 协议目前主要用于 “给 Agent 添加工具”。思考：MCP 还能用于什么场景？Agent-to-Agent 通信是否可以用 MCP？多个 Agent 共享同一个 MCP Server 会带来什么问题？